Daniel Suh

Software Engineer

Sacramento, CA | (916) 945-4584 | dsuh3508@gmail.com | linkedin.com/in/danielsuh8205 | github.com/dsuh02 | dsuh02.github.io

SUMMARY

I am a software engineer at Persist AI Formulations, where I own our lab information management system end to end and build the infrastructure under a multi-agent LLM product. I work primarily in PostgreSQL and FastAPI with React and TypeScript on top, and I run what I build: I operate the same databases and services locally, on on-prem Linux hosts, and in the cloud. Most of my recent work has been large-scale data migrations, retrieval over a million vectors behind measured quality gates, and production reliability.

EDUCATION & LEADERSHIP

University of Southern California

B.S. Computer Science · May 2025

CovidHacks 2020, Co-founder. Built the event site and led sponsorship outreach for an international virtual hackathon that drew 110+ participants and $9,400 in prizes (covidhacks2020.devpost.com).

EXPERIENCE

Persist AI Formulations

Sacramento, CA

Software Engineer · Scrum Master

May 2025 - Present

- Architected the company's lab operations platform end to end: an enum-safe PostgreSQL containment hierarchy with append-only immutable history, FastAPI endpoints for movement and state replay, and a React/TypeScript timeline for inspecting live containment trees. Built the instrument layer feeding it, including OPC UA servers and clients (`asyncua`), serial command bridges, and protocol schedulers driving real hardware. Later took its hot path from **roughly 100 database round trips to 2** by rewriting the bulk move endpoint as one modifying CTE.
- Consolidated **21 HTTP microservices into a single FastAPI process** coordinating an orchestrator and 17 specialist LLM agents, replacing cross-service calls with in-process tool dispatch. Removed every per-request network hop and reduced the full request path to one stack trace.
- Led migration of a **71-database data plane** (40 SQLite files and 19 on-disk vector indexes) onto a single **PostgreSQL/pgvector** cluster spanning 49 schemas, 388 tables, and roughly 14.5M rows. Validated in production with the legacy disk volume fully detached: 33-second clean boot, 15 of 15 vector stores reachable, 585 of 586 upstream calls at HTTP 200, and no filesystem fallbacks.
- Rebuilt the retrieval layer end to end with a structure-aware chunker, 22 corpora re-embedded at 3,072 dimensions (adding 121K chunks), and a two-stage search path feeding approximate HNSW recall into an exact float32 re-rank. Gated the cutover on **recall@10 of at least 0.95**, which held across all 16 migrated corpora, 14 of them at 1.000.
- Hardened production reliability. Traced connection-pool exhaustion to sessions held open across SSE streams and multi-second upstream calls, audited all **26 affected handlers**, and shipped pool timeouts plus session-scoped hot paths. Separately closed six boot-validation gaps that let `/health/ready` return 200 over a keyless or database-dead stack. Four adversarial review passes found zero regressions.
- Designed encryption at rest for client deployments: SQLCipher for relational stores and an AES-256-GCM envelope for index, model, and document artifacts, all keyed by one master key fetched at runtime. Extended it to column-level sealing through custom SQLAlchemy `TypeDecorators` across **578 columns in 32 databases**, verified 578 of 578 with retrieval quality unchanged.
- Built the client factory that all upstream model traffic routes through, supporting **three independent auth modes per provider** (static key, Azure Managed Identity, and no-auth gateway passthrough) so traffic can stay inside a customer's own cloud tenant. Migrated roughly 50 call sites off raw SDK constructors to clear a 93-error production 401 cluster, and fixed a recursive-lock deadlock that froze the event loop on the first token fetch.
- On the client-facing portal, integrated a rules-based pricing engine with the correctness properties billing requires: idempotency keys, a quoted-hash guard that returns `409` rather than charging a stale price, and order creation plus ledger charge committed atomically under a row lock. Migrated a legacy inventory domain model across the React/TypeScript and FastAPI stack behind four blocking CI gates (**382 backend and 783 frontend tests**), with Playwright validation catching two live production bugs.

Software Engineer Intern

Jan 2025 - May 2025

- Cut protocol design time by roughly **60%** with automation scripts that replaced manual experiment setup.
- Ported and modernized a legacy Python web application, adding microsphere size prediction and PubChem-backed compound-similarity search to shorten R&D iteration cycles.

USC projects: Spotted (Jan - Jun 2024), geographic data visualization on Google Maps APIs over MySQL, with AJAX-driven views for reload-free interaction and Java servlets handling data and file management. **Emotify** (Aug - Dec 2023), OpenAI and Spotify API integration, cookie-based session management, servlet-rendered dynamic views, and SQL stored procedures securing the authentication flow.

SKILLS

Languages: Python, TypeScript / JavaScript, Java, C / C++, SQL, Bash

Backend & Data: FastAPI, Flask/Jinja2, SQLAlchemy (async ORM, pooling, AsyncSession), Pydantic v2, PostgreSQL, pgvector, SQLite / SQLCipher, MySQL, Alembic, schema migrations, REST API design, WebSockets, SSE, httpx, asyncio

Frontend: React, Vite, TypeScript, Zustand, Tailwind, Recharts

AI & Retrieval: multi-agent orchestration, LLM tool calling, RAG, vector search (pgvector HNSW / IVFFlat, FAISS), embedding pipelines, OpenAI / Anthropic / Google SDKs, provider routing and cost tiering, recall and eval gates

Infrastructure & Testing: Docker, Azure (Container Apps, Managed Identity, Key Vault, API Management), CI/CD (Azure DevOps), Linux, SSH, Git, Nuitka, uv, OPC UA (`asyncua`), pytest, Playwright, Vitest, Agile/Scrum, code review